

PORTADA

Título: Tareas GForma - Sistema de Gestión de Tareas y Control de Presencia **Autor/a:** M. Barca **Ciclo/curso:** 2º DAM - Curso 2025/26 **Fecha:** Mayo 2026 **Tecnologías principales:** Node.js, Electron, JavaScript (Vanilla), SQL Server, HTML/CSS.

RESUMEN (Abstract)

Tareas GForma es una aplicación de escritorio desarrollada con Electron diseñada para gestionar tareas, clientes, presupuestos y llevar un control horario del personal. Está dirigida a pequeñas y medianas empresas o centros de formación que requieran centralizar sus operaciones diarias en una única plataforma rápida y fiable. El problema principal que resuelve es la dispersión de datos y el seguimiento de tiempos: proporciona una interfaz unificada donde los empleados registran sus entradas, pausas y salidas, al mismo tiempo que los coordinadores asignan tareas, facturan y supervisan la carga de trabajo. Sus funcionalidades clave son:

1. Control de presencia y fichajes.
 2. Gestión CRUD completa de Tareas, Clientes y Presupuestos.
 3. Generación y exportación de informes horarios (PDF, CSV).
 4. Sincronización en tiempo real sobre base de datos centralizada.
 5. Sistema multiusuario con control de sesión concurrente.
-

1. Motivación e introducción

1.1 Motivación

La principal motivación para hacer este proyecto es unificar múltiples herramientas ofimáticas (hojas de cálculo inconexas, calendarios y correos) en un solo software a medida. Cubre la necesidad de tener trazabilidad de cada hora trabajada y asociada a una tarea específica, lo cual es vital para calcular rentabilidades.

1.2 Contexto y problema a resolver

Antes del desarrollo de Tareas GForma, la situación típica implicaba registros de asistencia en papel o en un Excel compartido, lo cual generaba problemas de concurrencia y pérdida de datos. La solución aporta un registro inmutable en base de datos relacional y un control estricto de accesos.

1.3 Objetivos propuestos

General:

- Desarrollar un sistema de gestión empresarial (ERP básico) robusto en formato aplicación de escritorio.

Específicos:

- Implementar una interfaz de usuario clara y reactiva (Vanilla JS/CSS).
 - Lograr persistencia de datos segura en SQL Server.
 - Permitir multiusuario concurrente sin colisión de datos.
 - Proveer salidas útiles como reportes CSV/PDF.
-

2. Metodología, recursos y planificación

2.1 Metodología utilizada

Se ha empleado una metodología ágil (inspirada en Kanban) dividiendo el trabajo por historias de usuario y módulos funcionales: Módulo de Base de datos, Módulo de Autenticación, Módulo de Tareas y Vistas HTML.

2.2 Estimación de recursos

Hardware: PC de desarrollo con Windows 10/11, servidor local o máquina virtual Windows Server 2019 para la base de datos. **Software:** VSCode (IDE), MS SQL Server Management Studio, Node.js, Electron Builder, Jest (pruebas), Git/GitHub (control de versiones).

2.3 Planificación

- Hito 1: Diseño de la base de datos e implementación del backend Node (2 semanas).
 - Hito 2: Desarrollo de la UI en HTML/CSS y conexión IPC (3 semanas).
 - Hito 3: Pruebas, concurrencia, exportación de informes y refactorización (1 semana).
-

3. Tecnologías y herramientas

- **Lenguaje/s:** JavaScript, SQL, HTML, CSS.
 - **Framework/s:** Electron (framework para app de escritorio).
 - **Base de datos:** Microsoft SQL Server.
 - **Herramientas:** Visual Studio Code, Git, GitHub, Jest, Postman, SSMS.
 - **Librerías clave:** mssql, dotenv.
-

4. Análisis

4.1 Definición de requisitos

Funcionales: Autenticación de usuarios; Fichaje de entrada/salida; Listado, creación y edición de tareas y clientes; Generación de informes horarios. **No funcionales:** Seguridad de credenciales (.env); Interfaz responsiva y amigable; Arquitectura mantenible.

4.2 Modelo de datos

Se utiliza una base de datos relacional (SQL Server). Entidades principales:

- `gf_Clientes` : Almacena información de contacto y fiscal.
- `Tareas` : Tareas asignadas, horas estimadas y fechas.
- `TareasTiempos` : Imputaciones de horas a una tarea concreta.
- `ControlPresencia_Fichajes` : Registro inmutable de eventos horarios. **Justificación:** La BD relacional es perfecta para este caso debido a las fuertes dependencias transaccionales (no se puede borrar un cliente si tiene facturas o tareas asociadas). Garantiza integridad referencial.

4.3 Casos de uso

1. El empleado inicia sesión introduciendo su usuario.
 2. El empleado "ficha" entrada en el módulo de presencia.
 3. El administrador consulta la lista de clientes.
 4. El administrador crea una tarea y se la asigna a un técnico.
 5. El técnico imputa 2 horas de trabajo a la tarea asignada.
 6. El responsable filtra el informe de presencia del último mes.
 7. El responsable exporta los datos de asistencia a CSV.
-

5. Diseño

5.1 Diseño general / arquitectura

La aplicación sigue la arquitectura multiproceso de Electron: [Main Process] <--- IPC ---> [Renderer Process] El **Main Process** (Node.js) se encarga de la conexión a SQL Server mediante DAOs (Data Access Objects como `TareaModel`), lógica de negocio y acceso al sistema de archivos. El **Renderer Process** (HTML/JS/CSS) muestra la interfaz y se comunica con Main mediante un canal IPC seguro (Preload).

5.2 Diagrama de clases (UML)

(Debido al formato texto de la memoria, se resume la estructura principal de clases de acceso a datos).

- `ClienteModel` : `getAll(filtros)` , `getById(id)` , `create(cliente)` , `update(id, cliente)` , `delete(id)` .
- `TareaModel` : `getAll()` , `getById()` , `getTiempos()` , `registrarTiempo()` .
- `PersonalModel` : Acceso a la información de los empleados. Ambos interactúan con `db.js` que contiene el Singleton `getPool()` de la BD.

5.3 Mockups / capturas de interfaz

Las capturas detalladas del Dashboard y los paneles de presencia se encuentran implementadas en el código fuente (ver vistas como `index.html` e `informe-presencia.html`).

5.4 Decisiones de diseño importantes

1. **IPC seguro**: En lugar de habilitar `nodeIntegration`, se usan puentes seguros (`preload.js`).
 2. **Consultas Complejas vía Procedimientos Almacenados**: Las búsquedas avanzadas (e.g. `up_bp_Tareas_Select`) se delegan al motor SQL para maximizar el rendimiento.
-

6. Implementación

6.1 Estructura del proyecto

- `/src/models` : DAOs para acceso a la BD (Patrón Repository).
- `/src/controllers` : Controladores IPC que exponen métodos a la UI.
- `/src/database` : Gestión del pool de conexiones SQL Server.
- `/src/views` : Interfaz de usuario (HTML/CSS/Vanilla JS).
- `main.js` : Punto de entrada de Electron.

6.2 Componentes principales

- **Persistencia (DAOs):** Uso intensivo de `mssql` y parámetros sanitizados para prevenir Inyección SQL.
- **Controladores / Servicios:** Mapeo de canales (e.g. `ipcMain.handle('clientes:getAll', ...)`).
- **UI:** Arquitectura SPA (Single Page Application) controlada por Vanilla JS con sistema de Toasts para notificaciones.

6.3 Dificultades encontradas y soluciones

- **Problema:** Desincronización del formato de fechas al insertar en SQL Server desde JS.
 - **Solución:** Forzar `SET DATEFORMAT dmy` en las consultas previas a sentencias INSERT/UPDATE complejas y utilizar un parseo estricto `_toDateOrNull()`.
-

7. Multiusuario concurrente (OBLIGATORIO)

Identificación: Los usuarios se identifican en `inteco-login.html`. En el backend, `db.js` valida credenciales en SQL llamando a `checkIdentidad` (`SeguridadUnificada_Identidad_Select`). Al iniciar, el main process guarda el objeto sesión. **Concurrencia:** Al usar Microsoft SQL Server y el pool de conexiones de Node (`mssql`), el sistema maneja múltiples operaciones transaccionales al mismo tiempo usando aislamiento a nivel de base de datos. **Fallos de comunicación:** Si se cae la red, el módulo `db.js` captura el evento `pool.on('error')` e invalida la conexión. Las vistas del Renderer capturan este error IPC y muestran un mensaje al usuario en formato *Toast* ("Error al conectar").

8. Despliegue e instalación

8.1 Manual de instalación

Requisitos: Instalar dependencias mediante `npm install` en la carpeta del repositorio clonado. Arrancar el script de la BD en SQL Server.

8.2 Configuración

Es obligatorio crear el archivo `.env` en la raíz (basado en `.env.example`) y configurar: `DB_USER=...` y `DB_PASSWORD=...`. Esto permite no exponer contraseñas reales en GitHub.

8.3 Datos de ejemplo

Cargar el archivo `database_creation_script.sql` en SQL Server proporciona las tablas y los registros dummy necesarios.

8.4 Manual de usuario

1. Iniciar la aplicación (`npm start`).
 2. Login con un usuario autorizado.
 3. Usar el menú lateral para acceder a Tareas o Informe de Presencia.
 4. Para exportar registros a CSV, ir a "Informe Presencia", filtrar por fecha y pulsar "Exportar CSV".
-

9. Pruebas

9.1 Plan de pruebas

El objetivo es verificar que las capas de acceso a datos (Modelos) generan las consultas SQL correctamente bajo distintas condiciones de entrada (Unit Test con Mocks).

9.2 Tabla de casos de prueba

ID	Modelo	Objetivo de la Prueba	Resultado
1	Personal	getAll filtra correctamente empleados activos	PASS
2	Personal	getAll maneja ausencias de filtros	PASS
3	Cliente	getAll realiza consulta básica	PASS
4	Cliente	getAll inyecta condición LIKE por nombre	PASS
5	Cliente	getId retorna datos concretos	PASS
6	Tarea	getAll llama al procedimiento almacenado	PASS
7	Tarea	getId pasa correctamente el IdTarea	PASS
8	Tarea	delete ejecuta SP de borrado sin errores	PASS

Se aporta evidencia en los ficheros `.test.js` e integrados usando el framework **Jest**.

10. Resultados, conclusiones y vías futuras

10.1 Objetivos alcanzados

Se han cumplido todos los requerimientos:

- CRUD de datos funcional.
- UI completa y fácil de usar.
- Concurrencia sobre base de datos.
- Informes generados correctamente (CSV añadido sobre el UI original).
- Eliminación de credenciales hardcodeadas (uso de variables de entorno).

10.2 Conclusiones

El proyecto ha demostrado la robustez de Electron combinada con un motor pesado como SQL Server. Usar Vanilla JS ha sido un desafío frente a frameworks modernos, pero ha permitido un control absoluto del DOM y la optimización del rendimiento en la aplicación de escritorio.

10.3 Vías futuras

1. Empaquetar la aplicación con un servidor backend desacoplado (REST API) en lugar de conexión directa SQL desde Main.

2. Implementar notificaciones push cuando se asigne una nueva tarea a un empleado conectado.
 3. Creación de una versión web/PWA para uso desde móviles fuera de la oficina.
-

11. Glosario

- **IPC:** Inter-Process Communication, mecanismo nativo de Electron para intercambiar mensajes.
 - **DAO:** Data Access Object, patrón estructural usado en las clases Model.
 - **SPA:** Single Page Application, enfoque de desarrollo web donde solo se reescribe el cuerpo principal del HTML dinámicamente.
-

12. Bibliografía / webgrafía

- Documentación Oficial de Electron (<https://www.electronjs.org/>)
- Documentación de mssql npm package (<https://www.npmjs.com/package/mssql>)
- Jest Testing Framework (<https://jestjs.io/>)